Operating Systems

AIE 271

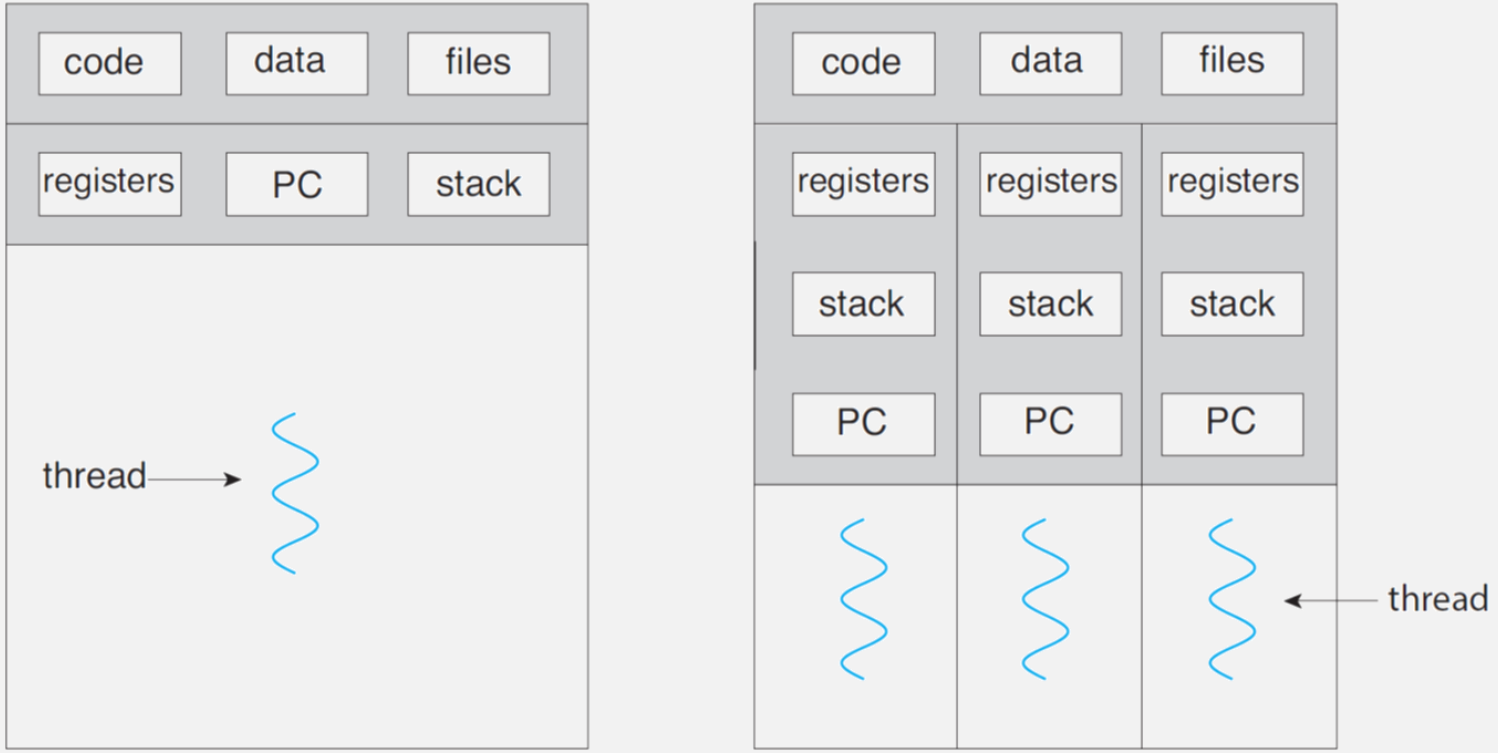
Lecture 4: Threads



Dr. Emad Badry

Thread Overview

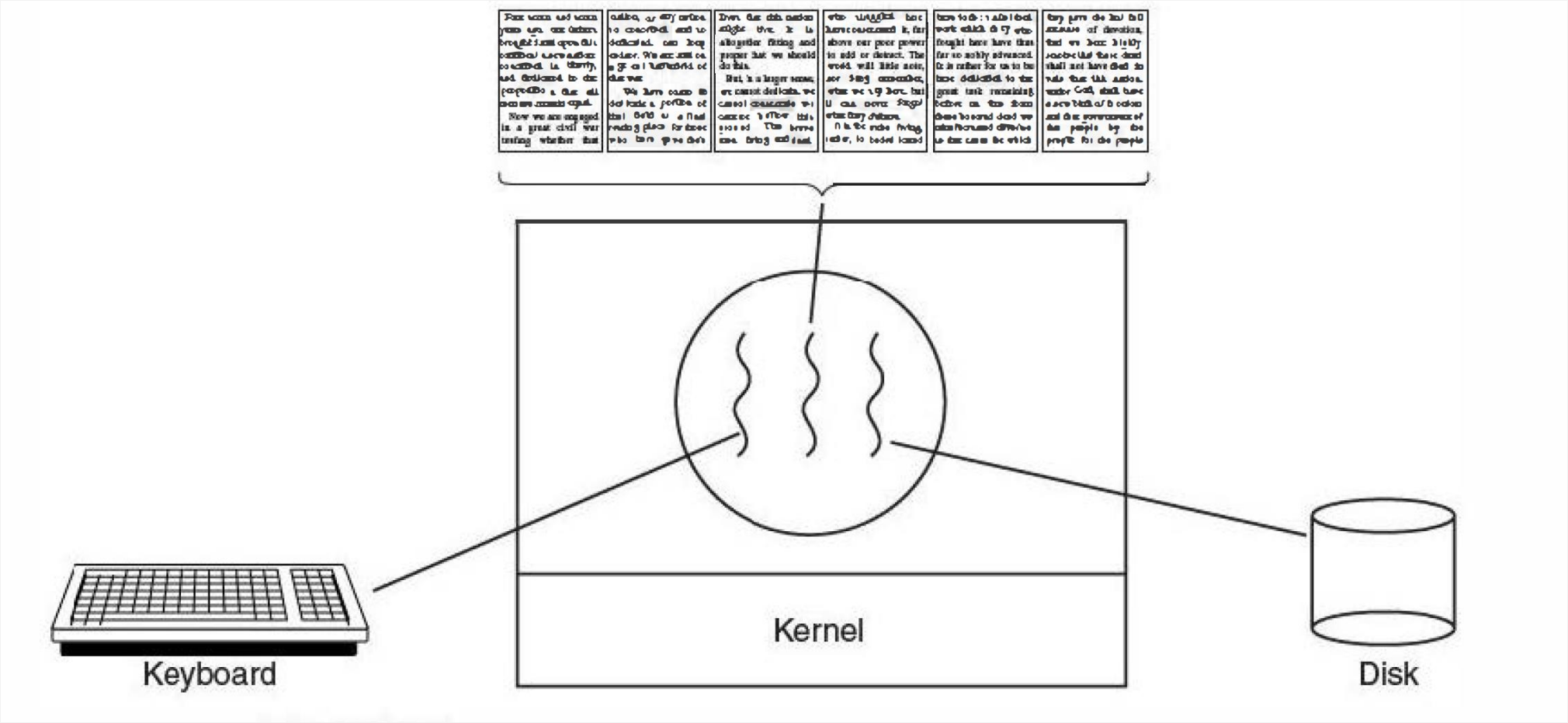
- A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter (PC), a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.



single-threaded process

multithreaded process

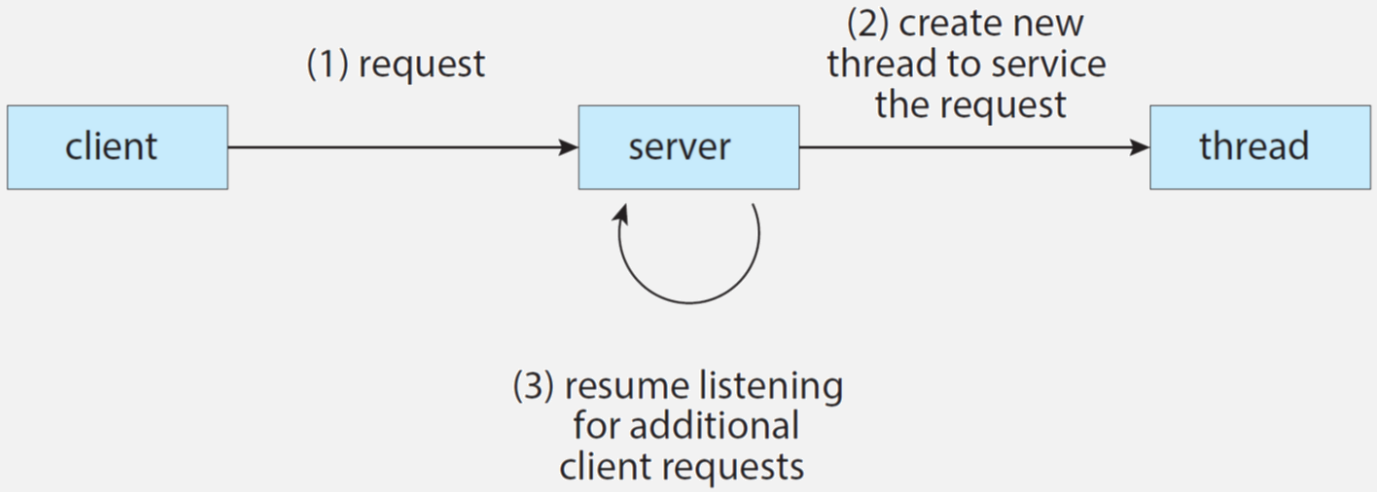
- Most software applications that run on modern computers and mobile devices are multithreaded. An application typically is implemented as a separate process with several threads of control.
- A web browser might have one thread display images or text while another thread retrieves data from the network.



- For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps thousands of) clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.

- One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request.

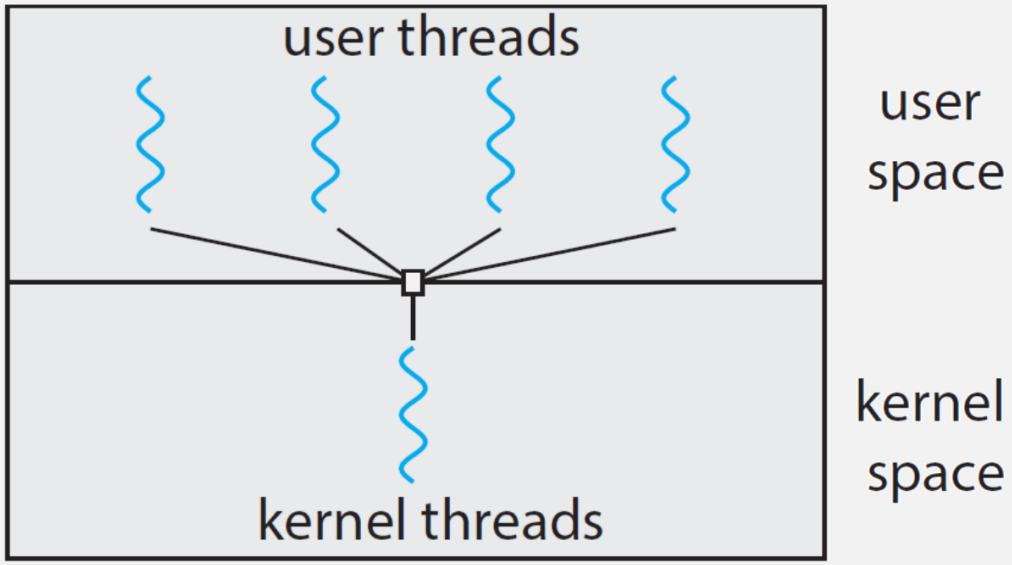
- If the web-server process is multithreaded, the server will create a separate thread that listens for client requests. When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests



Multithreading Models

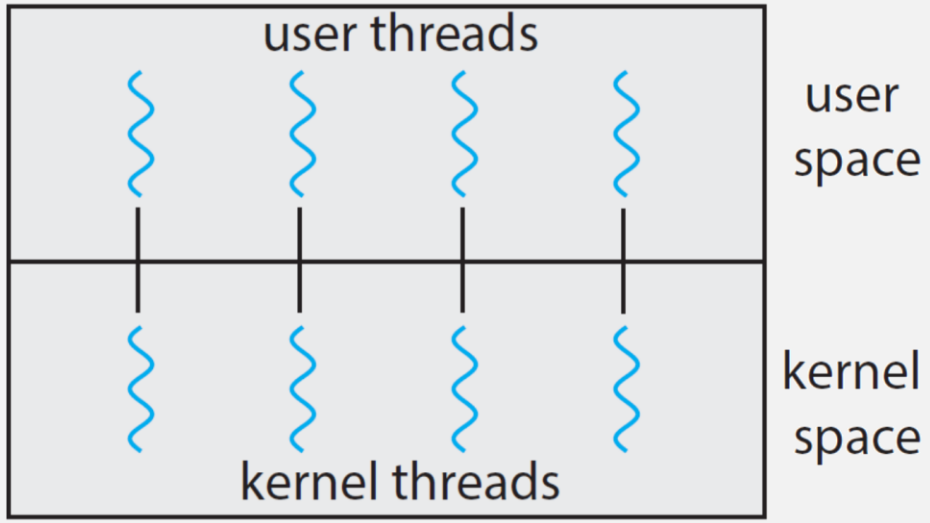
Many-to-One Model

- The many-to-one model maps many user-level threads to one kernel thread.
- Because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.



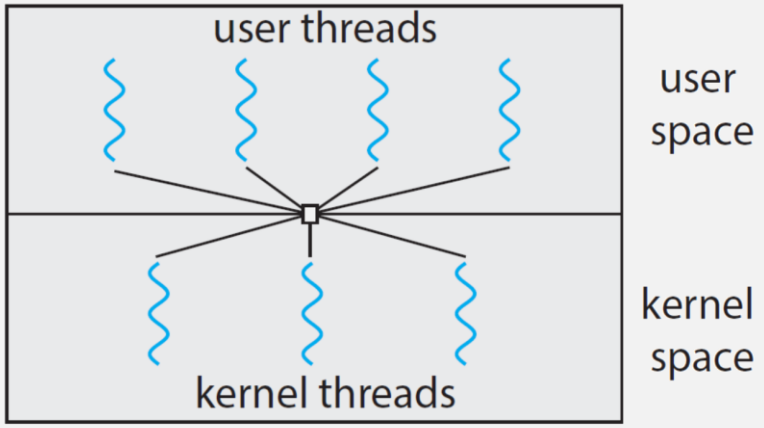
One-to-One Model

- The one-to-one model maps each user thread to a kernel thread.
- It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- It also allows multiple threads to run in parallel on multiprocessors.
- The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system.

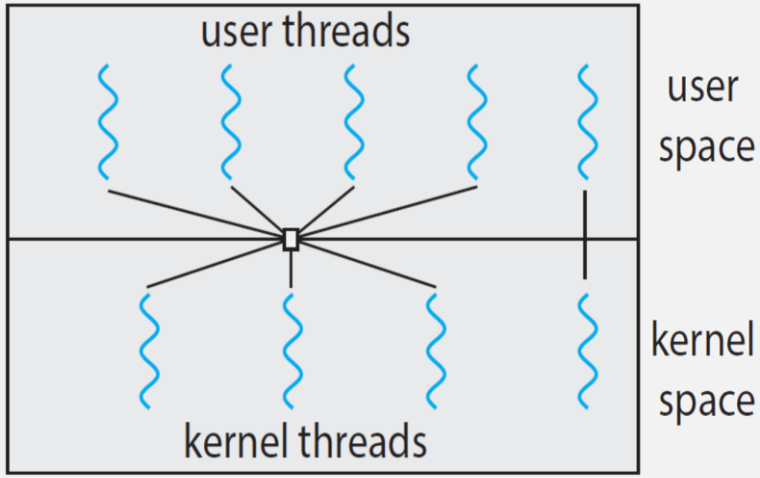


Many-to-Many Model

- The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.

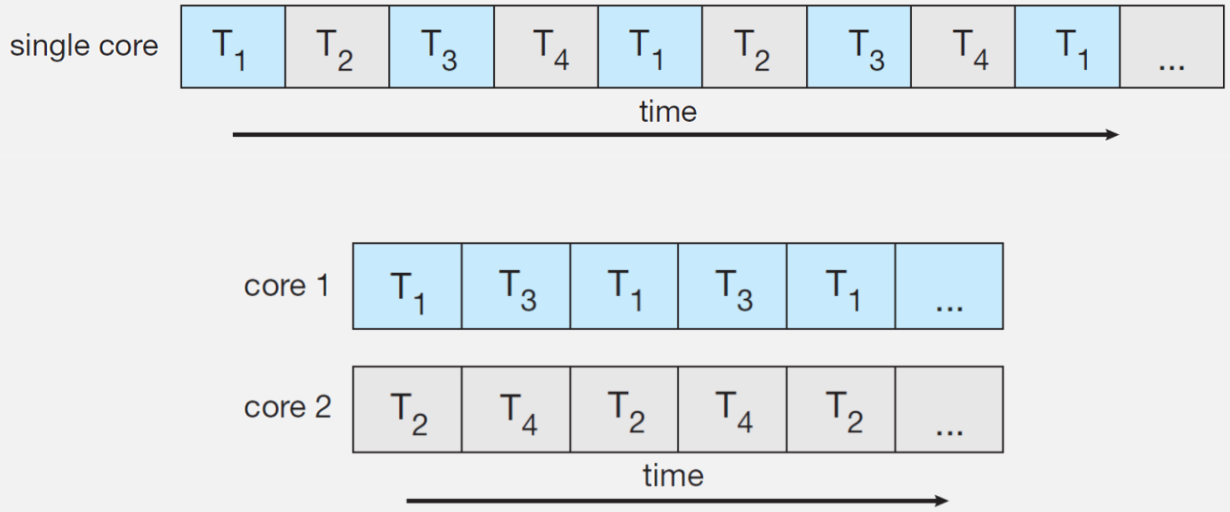


- One variation on the many-to-many model still multiplexes many user level threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread. This variation is sometimes referred to as the two-level model.



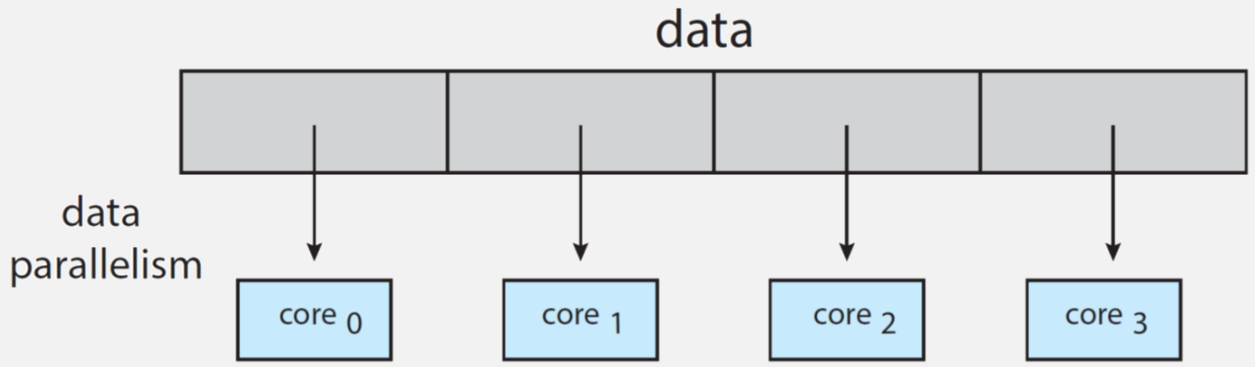
Multicore Programming

- On a system with a single computing core, concurrency merely means that the execution of the threads will be interleaved over time, because the processing core is capable of executing only one thread at a time.
- On a system with multiple cores, however, concurrency means that some threads can run in parallel, because the system can assign a separate thread to each core
- A concurrent system supports more than one task by allowing all the tasks to make progress. In contrast, a parallel system can perform more than one task simultaneously.



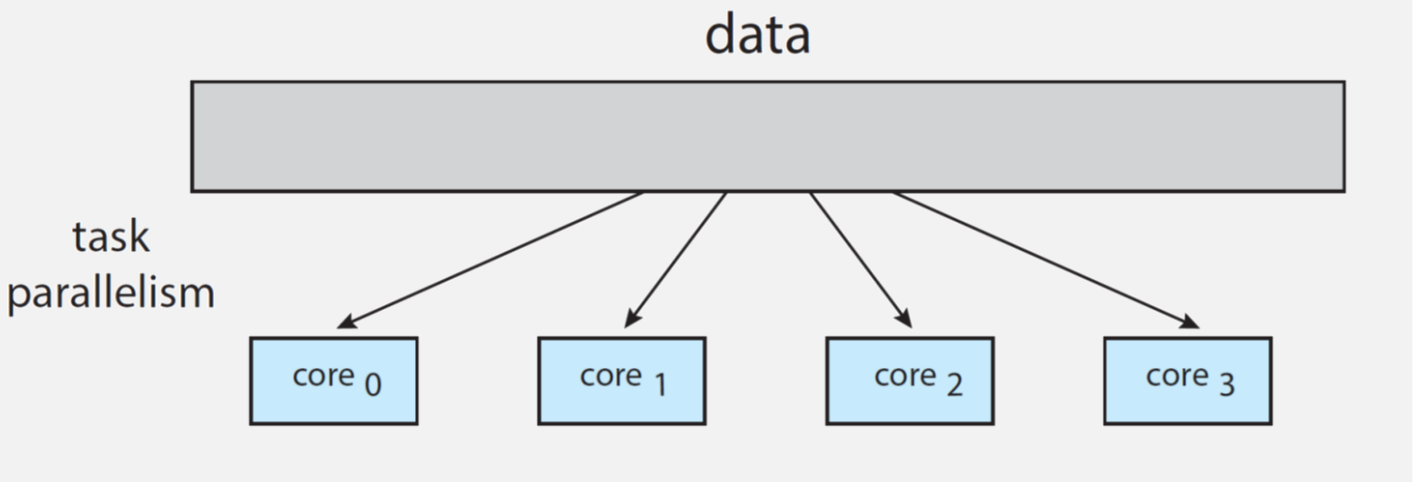
Types of Parallelism

- In general, there are two types of parallelism: data parallelism and task parallelism.
- Data parallelism focuses on distributing subsets of the same data across multiple computing cores and performing the same operation on each core.
- Consider, for example, summing the contents of an array of size N. On a single-core system, one thread would simply sum the elements [0] . . . [N − 1].
- On a dual-core system, however, thread A, running on core 0, could sum the elements [0] . . . [N / 2 − 1] while thread B, running on core 1, could sum the elements [N / 2] . . . [N − 1]. The two threads would be running in parallel on separate computing cores.



- Task parallelism involves distributing not data but tasks (threads) across multiple computing cores. Each thread is performing a unique operation. Different Threads may be operating on the same data, or they may be operating on different data.

- Fundamentally, data parallelism involves the distribution of data across multiple cores, and task parallelism involves the distribution of tasks across multiple cores.



Benefits of threads

Responsiveness.

Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.

Resource sharing

Threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

Economy

Thread creation consumes less time and memory than process creation. Additionally, context switching is typically faster between threads than between processes.

Scalability

Where threads may be running in parallel on different processing cores. A single-threaded process can run on only one processor, regardless how many are available.